

DETERMINISTIC LINTING, AUTHORED IN ENGLISH

RuleSmith

A rulebook a machine *runs* — not one an LLM *reads*

THE TRAP EVERYONE FALLS INTO

"Just put the rules in CLAUDE.md"

A CLAUDE.md rule is *prose pattern-matched by an LLM*. It has no control-flow graph, no call graph, no notion of lock state or call ordering. So it cannot answer the questions that catch the *expensive* bugs:

- *Is this `build()` called *before* a required setter? (typestate)*
- *Does a thread read-modify-write a shared field *without the lock*?*
- *Is this `@Pure` method actually pure?*

These aren't style nits. They're the bugs *SonarQube and PMD don't catch either* — and you certainly can't vibe them out of a sentence.

BEAT 1 — THE CORE ARGUMENT

Same code. Different flow. Opposite verdict.

`examples/java/PaymentGateway.java` — five defects no syntax rule can see.

```
$ rulesmith lint --rules $FIVE examples/java
warning[builder-terminal-before-setters]: 'b' finalized by build() before setter total()
warning[guarded-by-lock-held]: @GuardedBy field 'balance' read without the lock
warning[blocking-call-while-holding-lock]: fut.get() blocks while holding a lock
warning[pure-method-no-side-effects]: @Pure method calls a mutating method
warning[lambda-captures-mutable-state]: lambda mutates captured array 'acc'
5 findings

$ rulesmith lint --rules $FIVE examples/java-fixed
0 findings
```

The verdict flipped on the **flow**, not the syntax.

WHAT STANDARD LINTERS STRUCTURALLY CANNOT SEE

The differentiated ten

- **resource-leak** — `close()` on every path (post-dom)
- **builder-terminal-before-setters** — *typestate*: call ordering
- **atomic-get-set-race** — `get()`→`set()` defeats atomicity
- **guarded-by-lock-held** — `@GuardedBy` without the lock
- **blocking-call-while-holding-lock** — deadlock risk
- **command-query-separation** — returns *and* mutates
- **pure-method-no-side-effects** — purity via call graph
- **local-throw-catch-control-flow** — exception as goto
- **tell-dont-ask** — get-then-mutate from outside
- **lambda-captures-mutable-state** — closure capture

Typestate, lock dominance, purity, atomicity — **not** the textbook null-check every linter ships. These need a real CFG, call graph, and dataflow.

BEAT 2 — A REAL CATCH, NOT A TOY

A concurrency race in shipped code

```
$ rulesmith lint --rules atomic-get-set-race .../pdf/strategy/DefaultStrategy.java

warning[atomic-get-set-race]: Non-atomic get-then-set on Atomic 'textBB' loses concurrent updates.
--> DefaultStrategy.java:69
= note: value passed to textBB.set(...) is computed from textBB.get();
another thread can update between the get and the set.
= help: use textBB.updateAndGet(...) or compareAndSet(...)
```

`textBB.set(textBB.get() == null ? ... )` — a textbook lost-update race, in production, found deterministically. **No string rule finds this.**

WHY YOU CAN TRUST AN AI-WRITTEN RULE

AI builds the checks. Code runs them.

ACTION	NEEDS CLAUDE?	WHY
lint · list · --fix · CI gate	No	pure CFG + AST, deterministic, offline
add "<english>"	Once	codegen, then gated by fixtures
lint --judge	Cached	adjudicate, then cached → AI-free

The everyday workflow runs forever with **no API key**. Claude only *authors* a rule (gated by tests) and optionally *adjudicates* a borderline finding (cached).

BEAT 3 — THE CATALOG WRITES ITSELF

Describe a rule in English

```
$ rulesmith add "a switch over an enum must have a default case"
[attempt 1] compiling rule via claude -p...
[ok] violation.java: 1    [ok] clean.java: 0
installed rule 'enum-switch-default' (4 fixtures, all green)
```

The LLM writes the rule — a **deterministic fixture gate** decides if it ships.
That gate is exactly what a CLAUDE.md rule never gets. **114 rules** shipped this way.

BEAT 4 — IT POLICES ITSELF

Filters its own false positives

```
$ rulesmith lint --rules resource-leak --judge .../IcebergSourceTask.java
```

```
1 finding filtered as false positive by the judge:  
filtered[resource-leak] IcebergSourceTask.java:101: `reader` never closed  
= judge: OffsetStorageReader is framework-managed; Kafka Connect  
handles the lifecycle
```

Primitives are conservative on purpose; Claude adjudicates the residual, and the verdict is cached by `(rule, snippet)` so it's reproducible.

BEAT 5 — AND IT FIXES THEM

Safe autofix, the rest suggested

```
$ rulesmith lint --fix --dry-run --rules resource-leak .../OneDriveConnectorService.java
```

```
would fix: 2 resource(s) wrapped in try-with-resources
```

```
2 auto-fixed, 1 needs manual handling (suggest-only)
```

Only the **provably-safe subset** auto-fixes — a pure deterministic codemod, no AI.

Everything else stays a rustc-style suggestion with help + doc link.

BEAT 6 — DOES IT WORK ON REAL CODE?

Differentiated rules, real production files

114

RULES

1242

FILES SCANNED

0

PARSE ERRORS

The next slides are **real findings** from backend-connectors — the differentiated checks SonarQube doesn't run, firing on shipped code.

REAL CATCH · CONCURRENCY

atomic-get-set-race

PARSERS · DEFAULTSTRATEGY.JAVA:69

```
$ rulesmith lint --rules atomic-get-set-race examples/real-world/...  
warning[atomic-get-set-race]: Non-atomic get-then-set on Atomic 'textBB' loses concurrent updates.  
--> DefaultStrategy.java:69  
= note: value passed to set() is computed from get(); a thread can update between
```

A lost-update race a code reviewer would miss and a prose rule cannot express.

REAL CATCH · DESIGN

tell-dont-ask

FTP-PROBE · FTPCLIENTIMPL.JAVA:337

```
$ rulesmith lint --rules tell-dont-ask .../FtpClientImpl.java
warning[tell-dont-ask]: decide on nextReportTime.get() then nextReportTime.set() outside the object.
--> FtpClientImpl.java:337
= note: if (currentTime > nextReportTime.get()) { ... nextReportTime.set(...) }
```

Get-state-then-mutate from outside — the decision belongs *inside* the object.

REAL CATCH · EXCEPTIONS

local-throw-catch-control-flow

FTP-PROBE · FTPCONNECTORSERVICE.JAVA:415

```
$ rulesmith lint --rules local-throw-catch-control-flow ../FtpConnectorService.java
warning[local-throw-catch-control-flow]: 'IllegalStateException' thrown and caught locally in the same method,
steering control flow instead of signaling a fault.
--> FtpConnectorService.java:415
```

Exception used as a goto — `throw` post-dominated by its own `catch`.

REAL CATCH · DESIGN

command-query-separation

FTP-PROBE · FTPCLIENTIMPL.JAVA:72

```
$ rulesmith lint --rules command-query-separation ../FtpClientImpl.java
warning[command-query-separation]: Method 'create' returns a value and also mutates observable state.
--> FtpClientImpl.java:72
= help: split into a void command and a side-effect-free query
```

A query that secretly mutates — the bug class CQS exists to prevent.

REAL CATCH · RESOURCE LIFECYCLE

resource-leak

ONE-DRIVE · ONEDRIVECONNECTORSERVICE.JAVA:392

```
$ rulesmith lint --rules resource-leak .../OneDriveConnectorService.java
warning[resource-leak]: `outputStream` (OutputStream) is never closed
--> OneDriveConnectorService.java:392
= note: no close() call found and the resource does not escape the method
```

The flagship — post-dominance proves `close()` is on no path. And it `--fix` es.

THE WHOLE THING IN ONE LINE

CLAUDE.md is read. RuleSmith is run.

114 rules implemented

typestate · purity · concurrency beyond SonarQube

0 API keys to run

tree-sitter + CFG/dominance + claude -p · catalog mined from Checker Framework, Effective Java, NASA Power of Ten,
OWASP, FP literature